Lab 7: Timers and Timekeeping

In this lab, we'll work with the ***hardware timers*** available on the ***ATMega328P*** to do some timing-related tasks. Specifically, we'll calculate the ***theoretical*** timing of the timers based on ***clock frequency***, ***divider***, and ***timer resolution***, and then we'll measure those times in the Wokwi emulator and see how accurate our theoretical clock calculations are.

Setting Up Timer 1

For this lab, we'll use ***Timer 1***. There are a couple of reasons for this:

- Timer 1 is a ***16-bit*** timer, meaning it counts from ***0*** up to ***65535*** (i.e. $2^{16} - 1$). This makes it take much longer to ***overflow*** and makes it a lot easier to keep track of.
- The ***ATMega328P*** that is used by the ***Arduino*** only has one ***16-bit*** timer (Timer 1), unlike some more expensive AVR controllers, so on the Arduino specifically we only have one 16-bit timer option.
- For doing the IRL timing of our counter, we'll want to use the ***millis()*** function, which in the ***Arduino*** ecosystem is run off ***Timer 0***. We want to avoid interfering with that timer or our ***millis()*** function will return incorrect results.

Our first steps then will be to ***initialize Timer 1***. This involves setting ***two*** registers – the ***TCCR1A*** register (this register is used to configure special I/O options and overflow handling), and the ***TCCR1B*** register, which is used to set the ***prescaling clock divider*** of the timer.

The ***TCCR1A*** register will interfere with the ***overflow*** behavior of the timer, which will give the appearance of running too fast (compared to our theoretical calculations). So, we need to set this register to a value of ***0*** (all flag bits are zero) to prevent this.

The ***TCCR1B*** register will take a value between ***0*** and ***5***, indicating the ***binary value*** of the ***CS00***, ***CS01***, and ***CS02*** bits. These bits set the ***clock divider*** used to drive the timer, in accordance with the table shown below:

TCCRx			Synchronous Timer0 and Timer1 P_{CK} = CK
Bit 2	Bit 1	Bit 0	
CSx2	CSx1	CSx0	$T_{CK0/1}$
0	0	0	0 (Timer Stopped)
0	0	1	P_{CK} (System Clock)
0	1	0	$P_{CK}/8$
0	1	1	$P_{CK}/64$
1	0	0	$P_{CK}/256$
1	0	1	$P_{CK}/1024$
1	1	0	External Clock on Tn pin
1	1	1	External Clock on Tn pin

For example, writing a ***0*** to the ***TCCR1B*** register will ***stop the timer*** at its current count value.

Writing a ***3*** to the ***TCCR1B*** register will set ***CS00*** and ***CS01*** to ***1***, creating a timer with a ***/64 clock divider***.

Writing a ***5*** to the ***TCCR1B*** register will set ***CS02*** and ***CS00*** to ***1***, creating a timer with a ***/1024 clock divider***.

We will not be using an ***external*** clock source in this lab, but if we did then value ***6*** and ***7*** in ***TCCR1B*** would be based on the external clock.

The initial timer setup code is provided below – ***register 16*** is used to set the ***TCCR1A*** register to ***zero*** and we initialize the ***TCCR1B*** register to the clock pre-divider value. You will need to modify this code slightly to change the ***clock divider*** throughout the lab as we measure the different clock speeds.

```
extern "C" {
  void setup_timer(void);
  void run_timer(void);
  extern uint8_t prediv;
}

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
  prediv = 4; // Set clock predivider to /256
  setup_timer();
}

void loop() {
  // put your main code here, to run repeatedly:
}
```

```
.global setup_timer
.global run_timer
.global prediv

.data
prediv: .BYTE 5

.text
setup_timer:
push r16
ldi r16, 0
sts 0x80, r16 // Set TCCR1A to 0
lds r16, prediv
sts 0x81, r16 // Set TCCR1B to prediv
pop r16
ret
```

Note that we're also using ***serial***, and we're using it at the higher ***115200 baud rate*** instead of ***9600*** – this helps minimize the impact of the ***serial printing*** (very slow) on our timing calculations.

Theoretical Timing Values

Before we implement the actual timers to measure time, we'll get a yardstick via some simple calculations. The relevant equations are ***included in an Atmel Application Note***. This kind of information is almost always provided by the manufacturer in some form of ***datasheet*** or ***application note***. From the note:

$$TOV_{CK} = \frac{f_{CK}}{MaxVal} = \frac{(P_{CKx} / PVal)}{MaxVal} = \frac{P_{CKx}}{(PVal \bullet MaxVal)} \qquad (2)$$

Where:

- ***TOV_{CK}*** is the ***number of overflows the counter experiences in a second***
- ***P_{CKx}*** is the ***main clock frequency of the system*** (***16MHz*** for ***Arduino***)
- ***PVal*** is the ***divider value*** (i.e. ***1, 8, 64, 256…***)
- ***MaxVal*** is the maximum value of the counter (***65535*** for a ***16-bit***, ***255*** for ***8-bit***)

We have all the information required to calculate the ***theoretical*** time value for the counter overflow (i.e., how long does it take for the counter to count from ***0 to 65535***). We can then convert the number of overflows per second into a ***seconds-per-overflow*** value by simply dividing ***1*** by the resulting ***TOV_{CK} value***.

For example, with a ***main clock of 16MHz***, a ***divider of 256***, and using ***timer 1*** with a ***MaxVal*** of ***65535*** (16-bit timer):

16,000,000 / (256 * 65535) = 0.9536

We get a ***TOV_{CK}*** of ***0.9536***. When we divide ***1 / 0.9536***, we get ***1.048*** – it will take approximately ***1.048*** seconds for timer 1 to go from ***0*** to an ***overflow***, when using the ***256*** divider.

Calculate a full table of theoretical values as time-to-overflow (in seconds) for each divider value (include ***1***, no divider). Include this in a ***comment*** at the end of your assembly code when you submit.

Measuring Real-World Timing Values

Now that we've calculated the ***theoretical*** times, we can measure the ***actual*** timer overflow times to see how close the ***theoretical*** values are to reality.

The ***millis()*** function is a convenience function provided by the Arduino base library, which provides the number of ***milliseconds since the device was started***. This is a ***16-bit*** value that will eventually overflow, but it is useful for measuring ***elapsed time*** between two measurements.

```
void loop() {
  // put your main code here, to run repeatedly:
  run_timer();
  Serial.println(millis());
}

```

The code is required is provided at the right. This code checks the ***TIFR1*** overflow register. When the timer has overflowed, the first bit of ***TIFR1*** will be set to 1. We repeatedly check the value of ***TIFR1*** and check to see if this bit is set. If it isn't, we

```
run_timer:
push r16
push r17
push r18
loop:
ldi r16, 1
in r17, 0x16 // Load value from TIFR1
and r16, r17 // Check if TIFR1 overflow bit is set
tst r16 // If not set, go back to loop and try again
breq loop
ldi r18, 0
out 0x16, r18 // Reset the TIFR register to 0
pop r18
pop r17
pop r16
ret
```

branch backwards and continue to loop and check ***TIFR1***. If the overflow bit ***is*** set, we set the ***TIFR1*** register back to ***zero*** (so the overflow flag is no longer set) and return after popping our data off the stack. Since the ASM code ***loops*** internally until the overflow happens, it only performs the ***Serial.println()*** call when the ***overflow bit*** has been set. It then prints the current ***milliseconds elapsed since the device started***.

When we run this code, we'll get serial output that looks like that shown on the right.

```
1047
2096
3144
4193
5241
6290
```

This example was using a ***clock divider of 256***. We can see the first overflow happened ***1047 milliseconds*** (***1 millisecond = 0.001 seconds)*** after the device started. The next overflow happened at ***2096 milliseconds after starting***. Subtracting ***1047*** from ***2096***, we see the overflow took approximately ***1049 milliseconds,*** or ***1.049*** seconds.

You might notice there is a small bit of ***jitter*** on the results – for example, ***3144*** minus ***2096*** is ***1048 milliseconds*** (1.048 seconds) – so the timer isn't quite accurate to the ***millisecond***, there is a small amount of error.

If we compare against the ***theoretical value*** we calculated for the ***256 divider*** earlier, we can see they're ***very, very close*** (1.048 seconds calculated, 1.048-1.049 measured). Any error (+/- a millisecond) will be down to other parts of our code taking time (assembly loop, serial print, etc. – those all take a little bit of time).

Run your code for each of the different timer dividers. ***Calculate the actual elapsed time for each divider and include it in a table as a comment at the end of your code, alongside the theoretical values you calculated.*** These values should be quite similar – if they aren't, you're probably doing something wrong!

Theory Meets Reality – Estimate and Measure

As a final challenge (now that we've seen how the timers work), let's try an experiment – we'll clock our ***Arduino*** at a different, non-standard clock speed. Click on the ***diagram.json*** tab in ***Wokwi***, and modify your JSON so it looks like the file shown below:

sketch.ino ● diagram.json ● Library Manager

```
{
  "version": 1,
  "author": "Anonymous maker",
  "editor": "wokwi",
  "parts": [ {  "id":       "uno",
                "type":     "wokwi-arduino-uno",
                "attrs": {"frequency": "1m"}
            } ],
  "connections": []
}
```

Notice the ***frequency*** attribute that is being set for the ***Uno*** part.

This will change the default clock frequency of the board from ***16MHz*** to ***1MHz***. This means the

P_{CKx} of our calculations will be ***1,000,000*** instead of ***16,000,000***.

After modifying the main clock frequency, calculate the ***theoretical*** time required to overflow ***Timer 1*** when using a ***1024*** divider. Then, try simulating it in Wokwi, and measure the amount of time using ***millis()***. Determine the error between the ***theoretical value*** and the ***realworld value***. Non-square-of-two main frequencies (such as ***1MHz***) will introduce more error into the ***millis()*** function and the overall timekeeping of the system due to rounding errors, and longer time periods give more time for error to build up – so this value will deviate a bit more than the previous examples. Include the two values (calculated and real) and the error (difference) between them as a ***comment at the end of your code, after the tables.***

Lab Submission

When you're all finished, ***submit your code on Moodle***. Your code should have comments at the end including:

- Your initial calculated values for ***16MHz main frequency*** (all dividers), seconds per overflow
- Your actual values as calculated with ***millis()*** for each of the dividers
- Your calculated and real-world values for the ***1MHz*** main clock frequency and the difference between them

I recommend just copy-pasting all the code from the ***Wokwi*** editor into new files in ***Notepad++*** or a similar program. Don't forget to include ***comments*** on your code and include your ***name*** and ***student number*** at the top. When you've done that, submit your code on Moodle for marking.